
Scilab 01: Basics of Scilab

Objectives of this Tutorial

On successful completion of this tutorial, you should be able to

1. Some basic operations
2. Use the command window
3. Perform element-wise operations
4. Perform matrix operations
5. Write Scilab scripts
6. Write Scilab functions
7. Using loops
8. Import/Export data from file

Scilab ??

- Scilab is an open-source software platform for numerical computation and scientific engineering. It is widely considered a free alternative to MATLAB and is heavily used in academia and industry.

>Open Source??

>Difference with MATLAB??

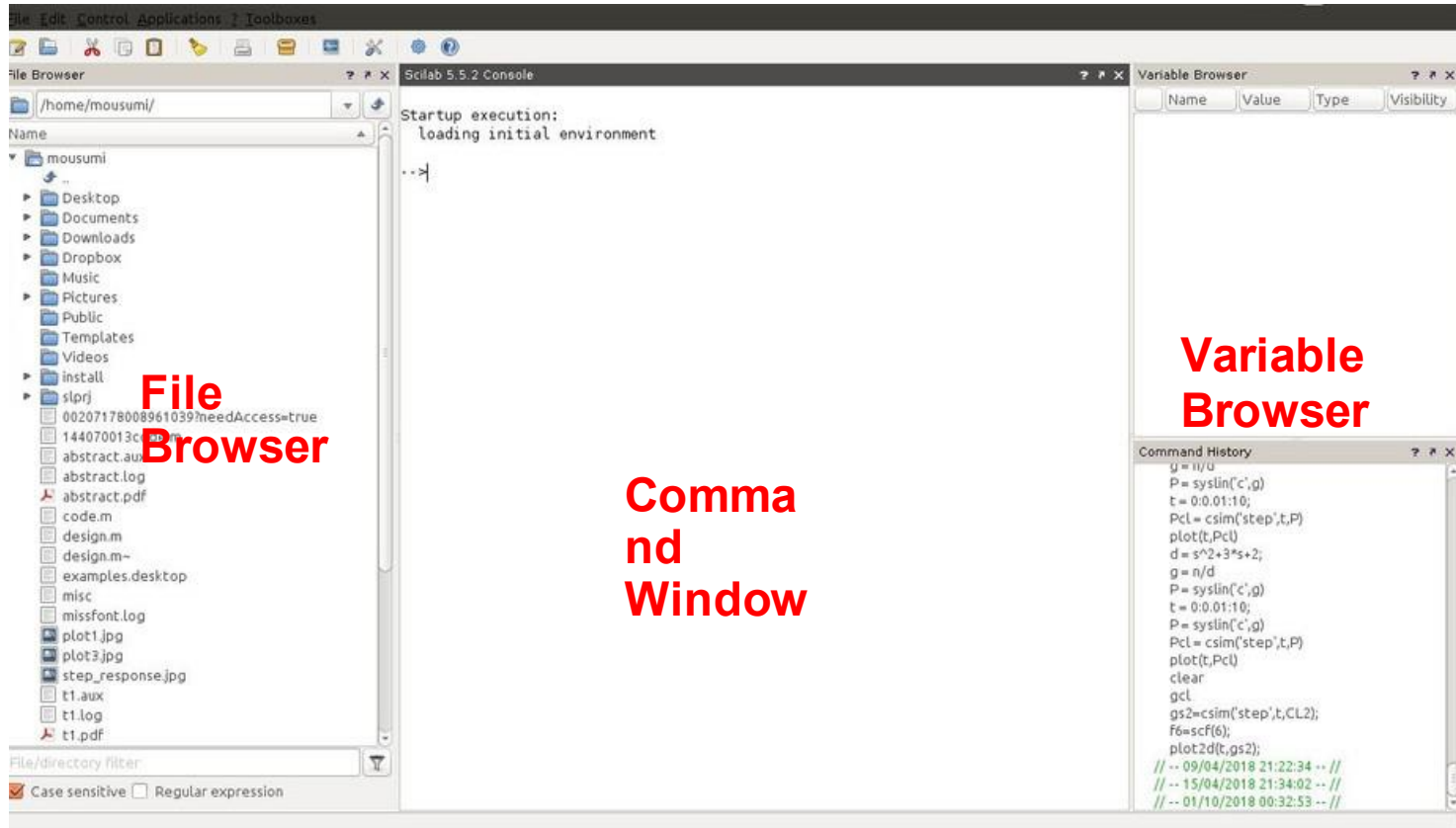
- Cost
- Performance
- Toolboxes

 **Which One Should You Choose?**

Setup Scilab and Work Environment

- Download Scilab:
<http://www.scilab.org/en/download/6.0.1>
or <https://www.scilab.org/>
- For Linux: Unzip tar.gz file at a destination.
Run \$
./Scilab_Location/scilab-6.0.1/bin/scilab
- For Windows: Install scilab-6.0.1_x64.exe

Get started



Command History

- **Console**
- **Command History Window**
- **Current Directory**
- **variable browser**
- **Script Files**
- **Executing a Script file**

Using the command window

Step1:Input two matrices A and B

Columns are separated by a space

Rows are separated by semicolon

Step 2: Input a vector x

```
--> A = [1 2 3;3 2 1;2 4 5]  
A =
```

```
1.    2.    3.  
3.    2.    1.  
2.    4.    5.
```

```
--> B = [5 4 2;1 8 9;5 4 1]  
B =
```

```
5.    4.    2.  
1.    8.    9.  
5.    4.    1.
```

```
--> x = [1;2;3]  
x =
```

```
1.  
2.  
3.
```

Scilab as a calculator

Example: --> $2+3/4*5$

Priority	Operation
1 (Highest)	Expressions inside parentheses ()
2	Exponentiation: ^ or **
3	Multiplication *, right division /, left division \ (evaluated left to right)
4 (Lowest)	Addition + and subtraction - (evaluated left to right)

Basic elements

Example: -->x=1

x = 1.

Variable Names: All ASCII letters from "a" to "z", "A" to "Z", "0" to "9" are allowed, with the additional letters "%", "_", "#", "!", "\$", "?".

Caution??

Allowed: NetCost, Left2Pay, x3, X3, z25c5

Not allowed: Net-Cost, 2pay, %x, @sign

Predefined mathematical variables

Variable Name	Description
<code>%pi</code>	the mathematical constant π
<code>%e</code>	Euler's constant e
<code>%i</code>	the imaginary number i

Strings

Example:

```
-->a="Mtech"  
a  =  
  
    "Mtech"  
  
-->b="First year"  
b  =  
  
    "First year"  
  
-->a + " " + b  
ans =  
  
    "Mtech First year"
```

Remarks:

- 1.Strings have no direct mathematical use.
- 2.They are used mainly for displaying a lingual message as a part of the output upon the execution of a program.

Dynamic nature of variable

It allows dealing with variables in a dynamic way.

```
-->x = 2 + 3 * %i
```

```
x =
```

```
2. + 3.i
```

```
-->y = 4*x
```

```
y =
```

```
8. + 12.i
```

Functions to manage complex numbers

Example:

```
-->z = 2 + 3 * %i
z =

    2. + 3.i

-->x = real(z)
x =

    2.

-->y = imag(z)
y =

    3.

-->isreal(z)
ans =

    F

-->
```

numbers.

Remark: Scilab distinguishes between real and purely real complex

Boolean

Comparison operators

Operator	Use	Description
==	a==b	returns truth value true if expression a is equal to b; otherwise false
~= or <>	a~=b or a<>b	returns truth value true if expression a is not equal to b; otherwise false
<	a<b	returns truth value true if a real expression a is less than b; otherwise false
>	a>b	returns truth value true if a real expression a is greater than b; otherwise false
<=	a<=b	returns truth value true if a real expression a is less than or equal to b; otherwise false
>=	a>=b	returns truth value true if a expression a is greater than or equal to b; otherwise false

Logical connective operators

Operator	Use	Description
&	A&B	logical AND operator: returns truth value true only for the case when both the logical expressions A and B have truth value true
	A B	logical OR operator: returns truth value true for when at least one of the logical expressions A and B have truth value true
~	~A	logical NOT operator: returns truth value true if the logical expressions A has truth value false and vice-versa

Example:

```
| -->a<>b  
| ans  =  
|  
| T  
|  
| -->a&b  
| ans  =  
|  
| T  
|  
| -->a|b  
| ans  =  
|  
| T  
|  
| -->~a  
| ans  =  
|  
| F
```

Elementwise matrix operations

```
--> A = [1 2 3;3 2 1;2 4 5]
```

```
A =
```

1.	2.	3.
3.	2.	1.
2.	4.	5.

```
--> B = [5 4 2;1 8 9;5 4 1]
```

```
B =
```

5.	4.	2.
1.	8.	9.
5.	4.	1.

```
--> x = [1;2;3]
```

```
x =
```

1.
2.
3.

```
--> Mat_Sum = A + B
```

```
Mat_Sum =
```

6.	6.	5.
4.	10.	10.
7.	8.	6.

```
--> Mat_Sub = A - B
```

```
Mat_Sub =
```

-4.	-2.	1.
2.	-6.	-8.
-3.	0.	4.

Elementwise matrix operations

```
--> A = [1 2 3;3 2 1;2 4 5]
```

```
A =
```

1.	2.	3.
3.	2.	1.
2.	4.	5.

```
--> B = [5 4 2;1 8 9;5 4 1]
```

```
B =
```

5.	4.	2.
1.	8.	9.
5.	4.	1.

```
--> x = [1;2;3]
```

```
x =
```

1.
2.
3.

```
--> Mat_element_product = A.*B
```

```
Mat_element_product =
```

5.	8.	6.
3.	16.	9.
10.	16.	5.

```
--> Mat_element_square = A.*A
```

```
Mat_element_square =
```

1.	4.	9.
9.	4.	1.
4.	16.	25.

```
--> Mat_scalar_product = 5*A
```

```
Mat_scalar_product =
```

5.	10.	15.
15.	10.	5.
10.	20.	25.

Elementwise matrix operations

```
--> A = [1 2 3;3 2 1;2 4 5]  
A =
```

```
1.  2.  3.  
3.  2.  1.  
2.  4.  5.
```

```
--> B = [5 4 2;1 8 9;5 4 1]  
B =
```

```
5.  4.  2.  
1.  8.  9.  
5.  4.  1.
```

```
--> x = [1;2;3]  
x =
```

```
1.  
2.  
3.
```

```
--> Mat_sqrt = sqrt(A)  
Mat_sqrt =
```

```
1.  1.4142136  1.7320508  
1.7320508  1.4142136  1.  
1.4142136  2.  2.236068
```

```
--> Mat_exp = exp(A)  
Mat_exp =
```

```
2.7182818  7.3890561  20.085537  
20.085537  7.3890561  2.7182818  
7.3890561  54.59815  148.41316
```

Matrix operations

```
--> A = [1 2 3;3 2 1;2 4 5]
```

```
A =
```

```
1.  2.  3.  
3.  2.  1.  
2.  4.  5.
```

```
--> B = [5 4 2;1 8 9;5 4 1]
```

```
B =
```

```
5.  4.  2.  
1.  8.  9.  
5.  4.  1.
```

```
--> x = [1;2;3]
```

```
x =
```

```
1.  
2.  
3.
```

```
--> Mat_trans = A'
```

```
Mat_trans =
```

```
1.  3.  2.  
2.  2.  4.  
3.  1.  5.
```

```
--> Mat_product = A*B
```

```
Mat_product =
```

```
22.  32.  23.  
22.  32.  25.  
39.  60.  45.
```

```
--> Mat_vec_product = A*x
```

```
Mat_vec_product =
```

```
14.  
10.  
25.
```

Matrix operations

```
--> A = [1 2 3;3 2 1;2 4 5]  
A =
```

```
1.    2.    3.  
3.    2.    1.  
2.    4.    5.
```

```
--> B = [5 4 2;1 8 9;5 4 1]  
B =
```

```
5.    4.    2.  
1.    8.    9.  
5.    4.    1.
```

```
--> x = [1;2;3]  
x =
```

```
1.  
2.  
3.
```

```
--> Mat_square = A*A  
Mat_square =
```

```
13.    18.    20.  
11.    14.    16.  
24.    32.    35.
```

```
--> Mat_power_3 = A^3  
Mat_power_3 =
```

```
107.    142.    157.  
85.     114.    127.  
190.    252.    279.
```

```
--> Mat_power_5 = A^5  
Mat_power_5 =
```

```
6721.    8938.    9907.  
5407.    7190.    7969.  
11938.    15876.    17597.
```

Length and size of matrix/vector

```
--> A = [1 2 3;3 2 1;2 4 5]
```

```
A =
```

```
1.  2.  3.  
3.  2.  1.  
2.  4.  5.
```

```
--> B = [5 4 2;1 8 9;5 4 1]
```

```
B =
```

```
5.  4.  2.  
1.  8.  9.  
5.  4.  1.
```

```
--> x = [1;2;3]
```

```
x =
```

```
1.  
2.  
3.
```

```
-->size_of_matrix = size(A)
```

```
size_of_matrix =
```

```
3.    3.
```

```
-->[row_size,column_size] = size(x)
```

```
column_size =
```

```
1.
```

```
row_size =
```

```
3.
```

```
-->total_elements = length(A)
```

```
total_elements =
```

```
9.
```

Range of matrices

```
--> A = [1 2 3;3 2 1;2 4 5]
```

```
A =
```

```
1.    2.    3.  
3.    2.    1.  
2.    4.    5.
```

```
--> B = [5 4 2;1 8 9;5 4 1]
```

```
B =
```

```
5.    4.    2.  
1.    8.    9.  
5.    4.    1.
```

```
--> x = [1;2;3]
```

```
x =
```

```
1.  
2.  
3.
```

```
-->Mat_element_31 = A(3,1)
```

```
Mat_element_31 =
```

```
2.
```

```
-->Mat_second_column = A(:,2)
```

```
Mat_second_column =
```

```
2.
```

```
2.
```

```
4.
```

```
-->Mat_first_two_column = A(:,1:2)
```

```
Mat_first_two_column =
```

```
1.    2.
```

```
3.    2.
```

```
2.    4.
```

```
-->Mat_third_row = A(3,:)
```

```
Mat_third_row =
```

```
2.    4.    5.
```

Range of matrices

```
--> A = [1 2 3;3 2 1;2 4 5]
```

```
A =
```

1.	2.	3.
3.	2.	1.
2.	4.	5.

```
-->Mat_Row23_Col12 = A(2:3,1:2)
```

```
Mat_Row23_Col12 =
```

3.	2.
2.	4.

```
--> B = [5 4 2;1 8 9;5 4 1]
```

```
B =
```

5.	4.	2.
1.	8.	9.
5.	4.	1.

```
--> x = [1;2;3]
```

```
x =
```

1.
2.
3.

Creates a submatrix of A.

Picks the elements of row2 to row 3 and col1 to col2 of A matrix

Construction of Special Matrices

```
--> A = [1 2 3;3 2 1;2 4 5]
```

```
A =
```

```
1.    2.    3.  
3.    2.    1.  
2.    4.    5.
```

```
--> B = [5 4 2;1 8 9;5 4 1]
```

```
B =
```

```
5.    4.    2.  
1.    8.    9.  
5.    4.    1.
```

```
--> x = [1;2;3]
```

```
x =
```

```
1.  
2.  
3.
```

```
-->Identity_3 = eye(3,3)
```

```
Identity_3 =
```

```
1.    0.    0.  
0.    1.    0.  
0.    0.    1.
```

```
-->Zero_32 = zeros(3,2)
```

```
Zero_32 =
```

```
0.    0.  
0.    0.  
0.    0.
```

```
-->Ones_45 = ones(4,5)
```

```
Ones_45 =
```

```
1.    1.    1.    1.    1.  
1.    1.    1.    1.    1.  
1.    1.    1.    1.    1.  
1.    1.    1.    1.    1.
```

Clear command window

Scilab 6.0.1 Console

```
--> A  
A =  
  
1.  2.  3.  
3.  2.  1.  
2.  4.  5.  
  
--> B  
B =  
  
5.  4.  2.  
1.  8.  9.  
5.  4.  1.  
  
--> clc
```

Scilab 6.0.1 Console

```
--> |
```

Clears command window

Clear variables

The image shows two instances of the Scilab 6.0.1 interface. The top instance shows a console with a prompt '--> |' and an empty Variable Browser. The bottom instance shows the same console with the command '--> clear' entered and highlighted by a blue box. A red arrow points from this box to the Variable Browser in the bottom instance, which is empty. Another red arrow points from the top Variable Browser to the bottom one, indicating a transition or comparison.

Scilab 6.0.1 Console

```
--> |
```

Variable Browser

Name	Value	Type	Visibility
A	3x3	Double	local
B	3x3	Double	local
Identity_3	3x3	Double	local
Mat_Sub	3x3	Double	local
Mat_Sum	3x3	Double	local
Mat_ele...	2	Double	local
Mat_ele...	3x3	Double	local
Mat_ele...	3x3	Double	local
Mat_exp	3x3	Double	local
Mat_firs...	3x2	Double	local
Mat_po...	3x3	Double	local
Mat_po...	3x3	Double	local
Mat_pro...	3x3	Double	local

Scilab 6.0.1 Console

```
--> clear
```

--> |

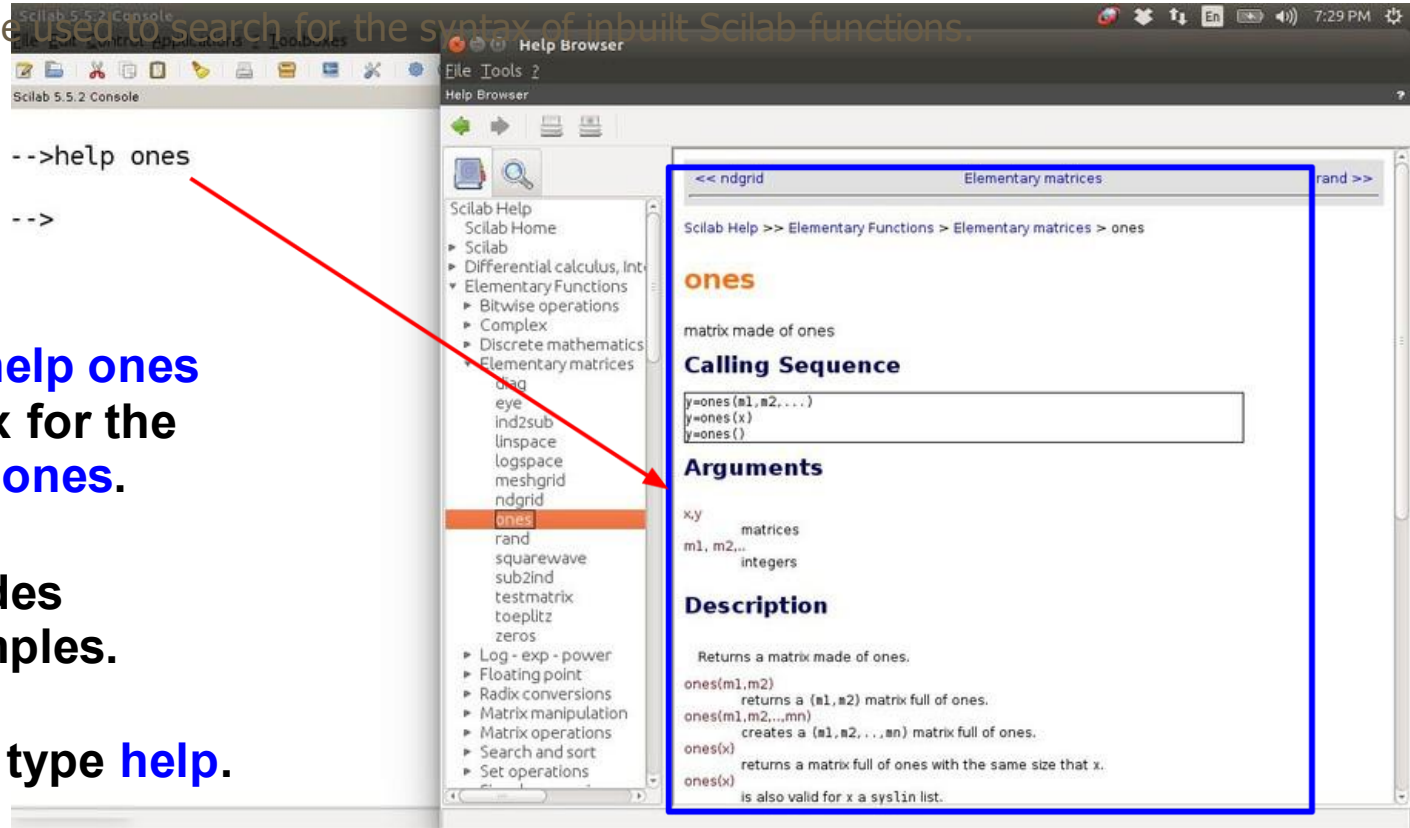
Variable Browser

Name	Value	Type	Visibility
------	-------	------	------------

Clears variables from the environment

Where to find help

Help command can be used to search for the syntax of inbuilt Scilab functions.



The command **help ones** gives the syntax for the Scilab function **ones**.

help also provides illustrative examples.

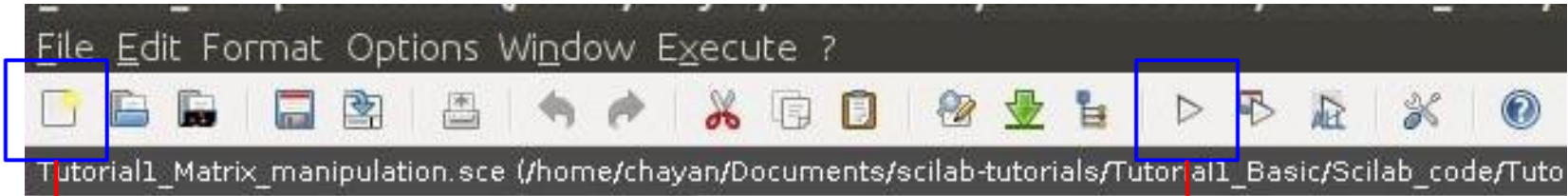
When in doubt, type **help**.

Scilab scripts:

File > Browse for new > Tutorial01-Basics/Scilab_code

Open file:

Tutorial1_Matrix_manipulation.sce



To create Scilab scripts

To execute Scilab scripts

A screenshot of the Scilab script editor window. The window title is 'Tutorial1_Matrix_manipulation.sce'. The script content is as follows:

```
1 //Script for Matrix manipulation tutorial
2 //This script gives a demonstration of a few basic matrix operations used in Scilab
3
4 //Manual entry of matrices (A,B,x)
5 A = [1 2 3;3 2 1;2 4 5];
6 B = [5 4 2;1 8 9;5 4 1];
7 x = [1;2;3];
```

Two red arrows point from the text 'Executable lines' to the first two lines of the script, which are commented out with '//'.

Executable lines

Symbol “//” used to write comments

Writing a Scilab script

Open file: **Tutorial1_function.sce**

```
//This Matlab script is used to compute mean and standard deviation of data
```

```
//Clears all previous variables stored  
clear
```

```
//Clears screen  
clc
```

```
//Executing the mymean function that computes the mean of a given data
```

```
exec mymean.sci;
```

```
//Executing the mystdev function that computes the standard deviation of a given data
```

```
exec mystdev.sci;
```

```
//Data for which the mean and standard-deviation is required
```

```
x = [1 2 5];
```

```
//Calling the mymean function. It takes data (a vector) as the input argument and returns mean of the data as output
```

```
mean_of_x = mymean(x);
```

```
disp(mean_of_x, 'Mean of x');
```

```
//Calling the mystdev function. It takes data (a vector) and its mean as the input argument
```

```
//It returns standard deviation of the data as output
```

```
stddev_of_x = mystdev(x, mean_of_x);
```

```
disp(stddev_of_x, 'Standard deviation of x');
```

Clears variables from the environment

Writing a Scilab script

```
//This Matlab script is used to compute mean and standard deviation of data
```

```
//Clears all previous variables stored  
clear
```

```
//Clears screen  
clc
```

Clears command window

```
//Executing the mymean function that computes the mean of a given data
```

```
exec mymean.sci;
```

```
//Executing the mystdev function that computes the standard deviation of a given data
```

```
exec mystdev.sci;
```

```
//Data for which the mean and standard-deviation is required
```

```
x = [1 2 5];
```

```
//Calling the mymean function. It takes data (a vector) as the input argument and returns mean of the data as output
```

```
mean_of_x = mymean(x);
```

```
disp(mean_of_x, 'Mean of x');
```

```
//Calling the mystdev function. It takes data (a vector) and its mean as the input argument
```

```
//It returns standard deviation of the data as output
```

```
stddev_of_x = mystdev(x, mean_of_x);
```

```
disp(stddev_of_x, 'Standard deviation of x');
```

Writing a Scilab script

```
//This Matlab script is used to compute mean and standard deviation of data
```

```
//Clears all previous variables stored  
clear
```

```
//Clears screen  
clc
```

```
//Executing the mymean function that computes the mean of a given data  
exec mymean.sci;  
//Executing the mystdev function that computes the standard deviation of a given data  
exec mystdev.sci;
```

```
//Data for which the mean and standard-deviation is required  
x = [1 2 5];
```

```
//Calling the mymean function. It takes data (a vector) as the input argument and returns mean of the data as output  
mean_of_x = mymean(x);  
disp(mean_of_x, 'Mean of x');
```

```
//Calling the mystdev function. It takes data (a vector) and its mean as the input argument  
//It returns standard deviation of the data as output  
stddev_of_x = mystdev(x, mean_of_x);  
disp(stddev_of_x, 'Standard deviation of x');
```

Calling the function mymean()

Variable passed: x

Return data saved in mean_of_x

Writing a Scilab script

```
//This Matlab script is used to compute mean and standard deviation of data
```

```
//Clears all previous variables stored  
clear
```

```
//Clears screen  
clc
```

To display the data in command window
Mean of x

```
//Executing the mymean function that computes the mean of x  
exec mymean.sci;  
//Executing the mystdev function that computes the standard deviation of x  
exec mystdev.sci;
```

2.6666667

```
//Data for which the mean and standard deviation is required  
x = [1 2 5];
```

```
//Calling the mymean function. It takes data (a vector) as the input argument and returns mean of the data as output  
mean_of_x = mymean(x);  
disp(mean_of_x, 'Mean of x');
```

```
//Calling the mystdev function. It takes data (a vector) and its mean as the input argument  
//It returns standard deviation of the data as output  
stddev_of_x = mystdev(x, mean_of_x);  
disp(stddev_of_x, 'Standard deviation of x');
```

Writing a Scilab script

```
//This Matlab script is used to compute mean and standard deviation of data
```

```
//Clears all previous variables stored  
clear
```

```
//Clears screen  
clc
```

```
//Executing the mymean function that computes the mean of x  
exec mymean.sci;  
//Executing the mystdev function that computes the standard deviation of x  
exec mystdev.sci;
```

```
//Data for which the mean and standard-deviation is required  
x = [1 2 5];
```

```
//Calling the mymean function. It takes data (a vector) as the input argument and returns mean of the data as output  
mean_of_x = mymean(x);  
disp(mean_of_x, 'Mean of x');
```

```
//Calling the mystdev function. It takes data (a vector) and its mean as the input argument  
//It returns standard deviation of the data as output
```

```
stddev_of_x = mystdev(x, mean_of_x);  
disp(stddev_of_x, 'Standard deviation of x');
```

Call a function and display

Standard deviation of x

2.081666

Writing a function

Open file: mymean.sci

To write a function we need:

- A function name: **mymean()**
- A function
 - Variable it takes as input (argument of the function): **data**
 - Return value: **mean_of_data**

//Function to compute mean of a given data (a vector)

function **mean_of_data** = **mymean**(**data**)

sum_of_data = 0;

for **iteration** = 1:1:**length**(**data**)

sum_of_data = **data**(**iteration**) + **sum_of_data**;

end

mean_of_data = **sum_of_data**/**length**(**data**);

endfunction

Writing a function

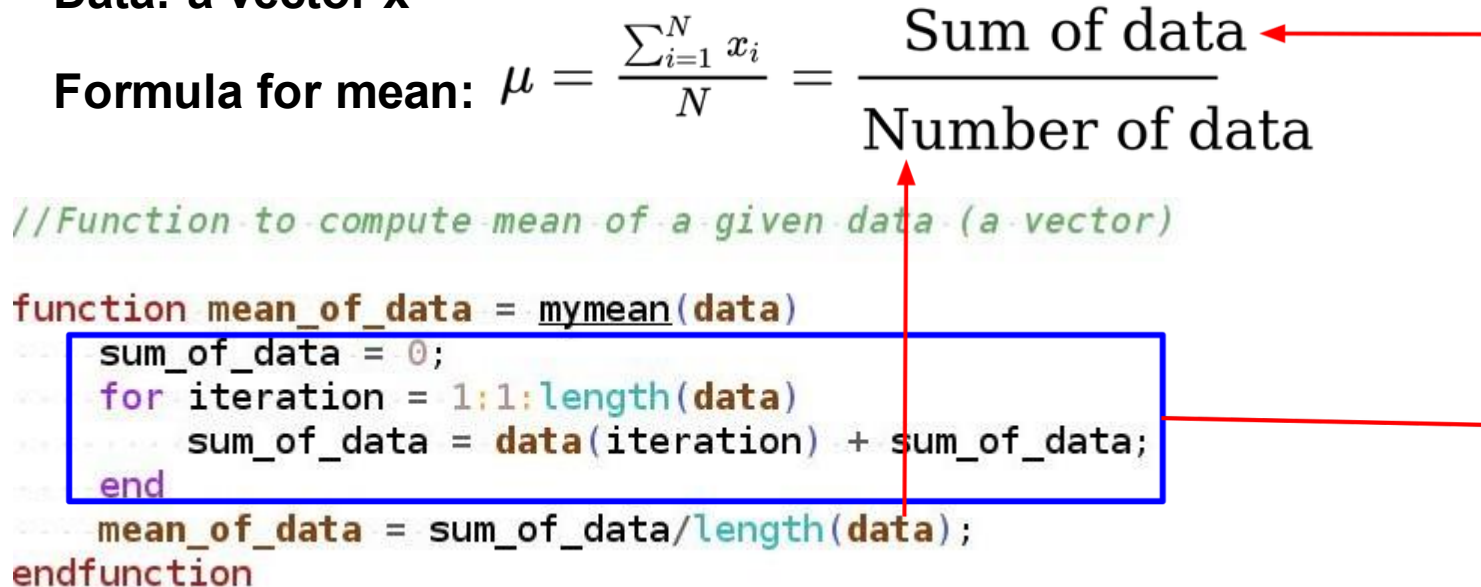
Objective: We write a function to compute mean

Data: a vector **x**

Formula for mean: $\mu = \frac{\sum_{i=1}^N x_i}{N} = \frac{\text{Sum of data}}{\text{Number of data}}$

//Function to compute mean of a given data (a vector)

```
function mean_of_data = mymean(data)
    sum_of_data = 0;
    for iteration = 1:length(data)
        sum_of_data = data(iteration) + sum_of_data;
    end
    mean_of_data = sum_of_data/length(data);
endfunction
```



Writing a loop

```
-->for variable = s:  
    // Instruction  
end|
```

```
-->while  
-->    di  
-->    i  
-->end
```

```
//Function to compute mean of a given data (a vector)
```

```
function mean_of_data = mymean(data)
```

```
    sum_of_data = 0;  
    for iteration = 1:1:length(data)  
        sum_of_data = data(iteration) + sum_of_data;  
    end
```

```
    mean_of_data = sum_of_data/length(data);  
endfunction
```

**Loop to sum the
elements of the
vector data**

Alternate: sum(data)

Writing the standard deviation function

Open file: `mystdev.sci`

Objective: We write a function to compute standard deviation

Data: a vector `x`, mean of `x`

Formula for standard deviation: $\sigma = \sqrt{\frac{\sum_{i=1}^N (x_i - \mu)^2}{N-1}}$

```
//A function to compute standard deviation of a given data  
//Input: Data (a vector) and mean of the data as input  
//Output: standard deviation of the data
```

```
function standard_dev = mystdev(data, mean_of_data)  
    temp = (data - mean_of_data*ones(length(data)))^2;  
    standard_dev = sqrt(sum(temp)/(length(data)-1));  
endfunction
```



File handling: importing data from csv files

```
//This function is to import data from a csv file and store it in a variable  
//Using the data mean and standard deviation of the data is computed  
clear  
clc
```

```
exec mymean.sci;  
exec mystdev.sci;
```

```
//Import data from csv file  
Data = csvRead(' ../Data/Tut1_data1.csv');  
x = Data(:,1);  
y = Data(:,2);
```

Variable_to_store_data = csvRead(filename)

```
//Compute mean of the imported data  
mean_of_x = mymean(x);  
//Compute standard deviation of the imported data  
stdev_of_x= mystdev(x,mean_of_x);
```

```
//Data to be exported (It can be a scalar, vector or matrix)  
data_to_write = [mean_of_x stdev_of_x];  
//Export data to a csv file  
csvWrite(data_to_write, ' ../Data/Result.csv')
```

File handling: importing data from csv files

```
//This function is to import data from a csv file and store it in a variable  
//Using the data mean and standard deviation of the data is computed  
clear  
clc
```

```
exec mymean.sci;  
exec mystdev.sci;
```

```
//Import data from csv file  
Data = csvRead(' ../Data/Tut1 data1.csv');  
x = Data(:,1);  
y = Data(:,2);
```



Segregating the data into variables

```
//Compute mean of the imported data  
mean_of_x = mymean(x);  
//Compute standard deviation of the imported data  
stdev_of_x = mystdev(x, mean_of_x);
```

The data is a matrix, in general

```
//Data to be exported (It can be a scalar, vector or matrix)  
data_to_write = [mean_of_x stdev_of_x];  
//Export data to a csv file  
csvWrite(data_to_write, ' ../Data/Result.csv')
```

After this use the data for analysis

File handling: mean and stdev of imported data

```
//This function is to import data from a csv file and store it in a variable  
//Using the data mean and standard deviation of the data is computed  
clear  
clc
```

```
exec mymean.sci;  
exec mystdev.sci;
```

```
//Import data from csv file  
Data = csvRead('..Data/Tut1_data1.csv');  
x = Data(:,1);  
y = Data(:,2);
```

```
//Compute mean of the imported data  
mean_of_x = mymean(x);  
//Compute standard deviation of the imported data  
stdev_of_x= mystdev(x,mean_of_x);
```

Computing mean and standard deviation of x

```
//Data to be exported (It can be a scalar, vector or matrix)  
data_to_write = [mean_of_x stdev_of_x];  
//Export data to a csv file  
csvWrite(data_to_write, '..Data/Result.csv')
```

File handling: exporting data to csv files

```
//This function is to import data from a csv file and store it in a variable  
//Using the data mean and standard deviation of the data is computed  
clear  
clc
```

```
exec mymean.sci;  
exec mystdev.sci;
```

```
//Import data from csv file  
Data = csvRead('..Data/Tut1_data1.csv');  
x = Data(:,1);  
y = Data(:,2);
```

```
//Compute mean of the imported data  
mean_of_x = mymean(x);  
//Compute standard deviation of the imported data  
stdev_of_x = mystdev(x, mean_of_x);
```

```
//Data to be exported (it can be a scalar, vector or matrix)  
data_to_write = [mean_of_x stdev_of_x];  
//Export data to a csv file  
csvWrite(data_to_write, '..Data/Result.csv')
```

Creating the variable to be written to a csv file

Variable is vector/matrix or scalar

File handling: exporting data to csv files

```
//This function is to import data from a csv file and store it in a variable  
//Using the data mean and standard deviation of the data is computed  
clear  
clc
```

```
exec mymean.sci;  
exec mystdev.sci;
```

```
//Import data from csv file  
Data = csvRead('.. /Data/Tut1_data1.csv');  
x = Data(:,1);  
y = Data(:,2);
```

```
//Compute mean of the imported data  
mean_of_x = mymean(x);  
//Compute standard deviation of the imported data  
stdev_of_x= mystdev(x,mean_of_x);
```

```
//Data to be exported (It can be a scalar, vector or matrix)  
data_to_write = [mean_of_x stdev_of_x];  
//Export data to a csv file  
csvWrite(data_to_write,'.. /Data/Result.csv')
```

Exporting variable to csv file

- **Variable to be written ,Filename**

File handling: other options

```
//This function is to import data from a csv file and store it in a variable
//Using the data mean and standard deviation of the data is computed
clear
clc
```

```
exec mymean.sci;
exec mystdev.sci;
```

```
//Import data from csv file
Data = csvRead('..Data/Tut1_data1.csv');
x = Data(:,1);
y = Data(:,2);
```

```
//Compute mean of the imported data
mean_of_x = mymean(x);
//Compute standard deviation of the imported data
stdev_of_x= mystdev(x,mean_of_x);
```

```
//Data to be exported (It can be a scalar, vector or matrix)
data_to_write = [mean_of_x stdev_of_x];
//Export data to a csv file
csvWrite(data_to_write, '..Data/Result.csv')
```

Other options of file handling

fscanfMat, fprintfMat

load, save